DESENVOLVIMENTO DE APLICATIVOS MOBILE



MÓDULO 05

Estado e Navegação

Estado em Compose, recomposição, state hoisting, efeitos colaterais, ciclo de vida e Navigation 3

Prof. Dr. Francisco
IFMA – Campus São Luís – Monte Castelo
Departamento de Eletroeletrônica (DEE)

1 de novembro de 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Criar estado com `remember` e `mutableStateOf` e entender a recomposição.
- Preservar estado em rotações com `rememberSaveable`.
- Aplicar *state hoisting* para criar componentes reutilizáveis e testáveis.
- Usar efeitos colaterais com segurança (`LaunchedEffect`, `DisposableEffect`...).
- Explicar o ciclo de vida da Activity e coletar fluxos respeitando-o.
- Implementar navegação entre telas com **Navigation 3**.

Roteiro do módulo

1. Estado e recomposição
2. State hoisting
3. Efeitos colaterais
4. Ciclo de vida
5. Navegação com Navigation 3
6. Fechamento

1
Estado e recomposição

O que é estado

Estado é qualquer valor que pode mudar com o tempo e afeta o que aparece na tela: o texto digitado, a lista carregada, se um diálogo está aberto, a última leitura do sensor.

Não funciona!

```
@Composable
fun Contador() {
  var cliques = 0          // variável comum
  Button(onClick = { cliques++ }) {
    Text("Cliques: $cliques")  // nunca muda
  }
}
```

Erro comum

Mudar uma variável comum não avisa o Compose, e a cada recomposição ela volta a 0.

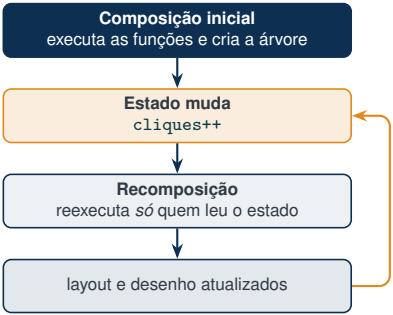
Correto

```
@Composable
fun Contador() {
  var cliques by remember { mutableStateOf(0) }
  Button(onClick = { cliques++ }) {
    Text("Cliques: $cliques")
  }
}
```

Duas peças

- `mutableStateOf`: valor **observável**; ler registra dependência, escrever agenda recomposição.
- `remember`: guarda o objeto entre recomposições.

Recomposição



Propriedades

- É **inteligente**: funções cujos parâmetros não mudaram podem ser puladas.
- Pode acontecer **a cada quadro** (ex.: animações).
- Pode ser **cancelada** e refeita.

Dica

O *Layout Inspector* mostra quantas vezes cada composable recompôs. Números muito altos indicam problema de desempenho (Módulo 11).

remember *versus* rememberSaveable

Sobrevivendo à rotação

```
// perdido ao girar a tela
var filtro by remember { mutableStateOf("") }

// salvo no Bundle: sobrevive à rotação e à
// morte do processo em segundo plano
var busca by rememberSaveable { mutableStateOf("") }

// estado derivado: só recalcula quando muda
val visiveis by remember(itens) {
  derivedStateOf { itens.filter { busca in it.descricao
      } }
}

// listas observáveis
val selecionados = remember {
      mutableStateListOf<String>() }
```

	recomposição	rotação
variável comum	perde	perde
`remember`	mantém	perde
`rememberSaveable`	mantém	mantém
ViewModel	mantém	mantém

Atenção

`rememberSaveable` só aceita tipos que cabem num `Bundle` (primitivos, `String`, `Parcelable`). Para objetos maiores, use um ViewModel (Módulo 06).

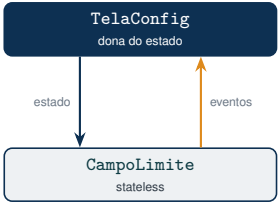
2
State hoisting

Elevando o estado (*state hoisting*)

Componente sem estado (stateless)

```
@Composable
fun CampoLimite(
  valor: String,                    // estado desce
  onValorChange: (String) -> Unit, // evento sobe
  modifier: Modifier = Modifier
) {
  OutlinedTextField(
    value = valor,
    onValueChange = onValorChange,
    label = { Text("Limite (°C)") },
    modifier = modifier
  )
}

// Quem usa decide onde o estado mora
@Composable
fun TelaConfig() {
  var limite by rememberSaveable { mutableStateOf("30") }
  CampoLimite(limite, onValorChange = { limite = it })
}
```



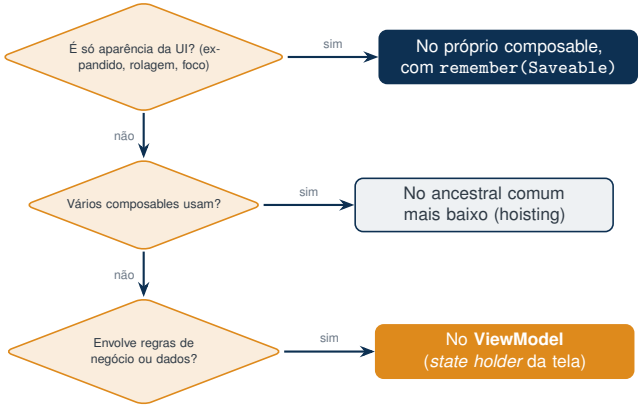
Fluxo unidirecional de dados (UDF)

O estado desce como parâmetro e os eventos sobem como lambdas. Há uma única **fonte da verdade**.

Vantagens

Componente reutilizável, fácil de testar e de visualizar em `@Preview`.

Onde o estado deve morar?



Dica

Regra prática: eleve o estado só até onde for necessário, nem mais nem menos.

3
Efeitos colaterais

APIs de efeitos

LaunchedEffect e DisposableEffect

```
@Composable
fun Monitor(bancadaId: Int, vm: PainelViewModel) {
  // roda quando entra na tela e quando a chave muda
  LaunchedEffect(bancadaId) {
    vm.carregar(bancadaId)   // coroutine
  }

  val lifecycle =
      LocalLifecycleOwner.current.lifecycle
  DisposableEffect(lifecycle) {
    val obs = LifecycleEventObserver { _, e ->
      if (e == Lifecycle.Event.ON_RESUME)
      vm.reconectar()
    }
    lifecycle.addObserver(obs)
    onDispose { lifecycle.removeObserver(obs) }
  }
}
```

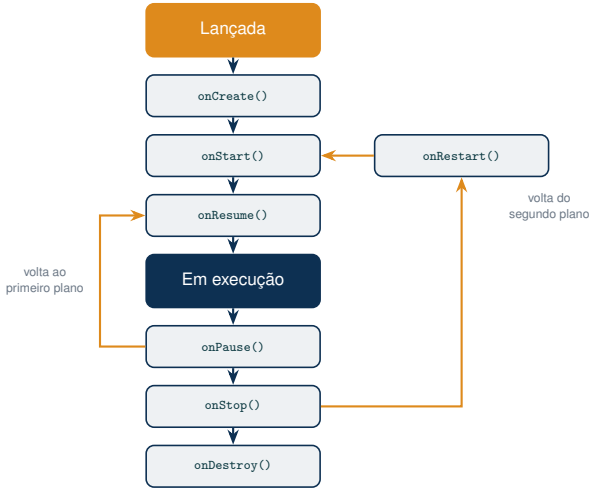
API	Quando usar
`LaunchedEffect(chave)`	rodar `suspend` ao entrar ou quando a chave muda
`rememberCoroutineScope`	iniciar coroutine a partir de um clique
`DisposableEffect`	registrar e remover *listeners*
`SideEffect`	sincronizar com código não-Compose a cada recomposição
`rememberUpdatedState`	usar o valor mais recente dentro de um efeito longo
`produceState`	converter fonte externa em estado
`derivedStateOf`	estado calculado que muda menos que a origem

Atenção

`LaunchedEffect(Unit)` roda uma única vez enquanto o composable estiver na tela.

4
Ciclo de vida

Ciclo de vida da Activity



Estado	Significado
Created	criada, ainda invisível
Started	visível
Resumed	em primeiro plano, recebe toques
Paused	parcialmente encoberta
Stopped	invisível (segundo plano)

Mudanças de configuração

Girar a tela, trocar idioma ou tema **destrói e recria** a Activity. Por isso o estado precisa estar em `rememberSaveable` ou no ViewModel.

Morte do processo

Em segundo plano, o sistema pode encerrar o app para liberar memória. Ao voltar, só o estado salvo sobrevive.

Coletando fluxos com segurança

TelaPainel.kt

```
@Composable
fun TelaPainel(vm: PainelViewModel = viewModel()) {
  // para de coletar quando o app vai para o
  // segundo plano (abaixo de STARTED)
  val estado by vm.estado.collectAsStateWithLifecycle()

  when (val e = estado) {
    PainelUiState.Carregando -> CircularProgressIndicator()
    is PainelUiState.Erro -> Text(e.mensagem)
    is PainelUiState.Pronto -> Painel(
      leituras = e.leituras,
      onAtualizar = vm::atualizar
    )
  }
}
```

`collectAsStateWithLifecycle`

Converte `StateFlow` em `State` do Compose e **pausa a coleta** quando a tela não está visível, economizando bateria, rede e sensores.

Dependência

Vem do artefato `androidx.lifecycle:lifecycle-runtime-compose`.

Erro comum

`collectAsState()` continua coletando em segundo plano. No Android, prefira a versão *WithLifecycle*.

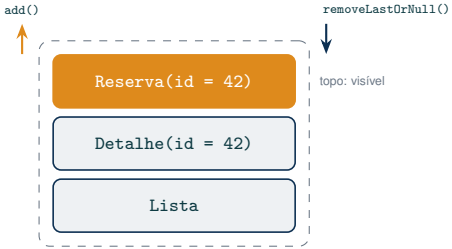
5
Navegação com Navigation 3

Conceitos de navegação

- Apps Compose costumam ter **uma Activity** e várias telas (*destinos*).
- A **pilha de navegação** (*back stack*) guarda o histórico; o botão/gesto voltar remove o topo.
- **Navigation 3** (Jetpack, estável desde nov./2025) representa a pilha como uma **lista observável de chaves** controlada pelo próprio app.

Peças

`NavKey` (chave de cada tela) · `rememberNavBackStack` (a pilha) · `NavDisplay` (exibe o topo) · `entryProvider` (chave → tela).



Dica

Muitos projetos existentes usam **Navigation Compose** (Nav2, com `NavHost` e `NavController`). Os conceitos são os mesmos; ao final mostramos a equivalência.

Navigation 3: chaves e pilha

app/build.gradle.kts

```
plugins {
  alias(libs.plugins.kotlin.serialization)
}
dependencies {
  implementation(libs.androidx.navigation3.runtime)
  implementation(libs.androidx.navigation3.ui)
  implementation(
    libs.androidx.lifecycle.viewmodel.navigation3)
  implementation(libs.kotlinx.serialization.core)
}
```

Rotas.kt

```
@Serializable data object Lista : NavKey
@Serializable data class Detalhe(val id: Int) : NavKey
@Serializable data class Reserva(val id: Int) : NavKey
```

Chaves tipadas

Cada tela é uma classe `@Serializable` que implementa `NavKey`. Os argumentos são propriedades: sem strings de rota e sem erros de digitação.

Pilha persistente

`rememberNavBackStack(Lista)` cria a pilha com a tela inicial e a salva automaticamente em rotações e na morte do processo, por isso as chaves são serializáveis.

Dica

Passe só identificadores (`id`) nas chaves; a tela de destino busca os dados.

Navigation 3: NavDisplay

NavegacaoApp.kt

```
@Composable
fun NavegacaoApp() {
  val pilha = rememberNavBackStack(Lista)
  NavDisplay(
    backStack = pilha,
    onBack = { pilha.removeLastOrNull() },
    entryDecorators = listOf(
      rememberSaveableStateHolderNavEntryDecorator(),
      rememberViewModelStoreNavEntryDecorator()), // VM por tela
    entryProvider = entryProvider {
      entry<Lista> {
        TelaLista(onAbrir = { id -> pilha.add(Detalhe(id)) })
      }
      entry<Detalhe> { chave ->
        TelaDetalhe(chave.id,
          onReservar = { pilha.add(Reserva(chave.id)) })
      }
      entry<Reserva> { chave -> TelaReserva(chave.id) }
    }
  )
}
```

- **Navegar** = `pilha.add(chave)`.
- **Voltar** = `pilha.removeLastOrNull()`.
- O gesto de voltar do sistema chama `onBack`.
- Os decoradores salvam o estado de cada tela e criam um ViewModel por entrada da pilha.

Telas não conhecem a pilha

`TelaLista` recebe `onAbrir` como lambda, não a pilha. Assim ela continua reutilizável e testável.

Navegação principal com barra inferior

Abas principais

```
enum class Aba(val chave: NavKey, val rotulo: String,
               val icone: ImageVector) {
  PAINEL(Painel, "Painel", Icons.Default.Dashboard),
  EQUIP(Lista, "Equipamentos", Icons.Default.Memory),
  CONFIG(Config, "Ajustes", Icons.Default.Settings)
}

NavigationBar {
  Aba.entries.forEach { aba ->
    NavigationBarItem(
      selected = pilha.first() == aba.chave,
      onClick = {           // troca a raiz da pilha
        pilha.clear(); pilha.add(aba.chave)
      },
      icon = { Icon(aba.icone, null) },
      label = { Text(aba.rotulo) }
    )
  }
}
```

Uma pilha por aba

Em apps maiores, mantenha uma pilha para cada aba e exiba a da aba ativa. Assim cada aba lembra até onde o usuário navegou. Os *recipes* oficiais do Navigation 3 trazem esse padrão pronto.

Nav2 ↔ Nav3

`NavHost`	`NavDisplay`
`navigate(r)`	`pilha.add(r)`
`popBackStack()`	`removeLastOrNull()`
`composable<R>{}`	`entry<R>{}`
`toRoute<R>()`	a própria chave

6
Fechamento

Resumo

Resumo

- `mutableStateOf` + `remember` criam estado observável; mudar o estado dispara a recomposição.
- `rememberSaveable` sobrevive à rotação; o ViewModel guarda o estado da tela.
- *State hoisting*: estado desce, eventos sobem, uma fonte da verdade.
- Efeitos colaterais só dentro de `LaunchedEffect`, `DisposableEffect` e afins.
- Rotação recria a Activity; colete fluxos com `collectAsStateWithLifecycle`.
- Navigation 3: chaves `NavKey`, pilha observável e `NavDisplay`.

Exercícios

Exercício 1 — Rotação

Crie um formulário com três campos, um usando `remember` e dois com `rememberSaveable`. Gire o emulador e explique o que aconteceu.

Exercício 2 — Hoisting

Transforme um `SeletorDeFaixa` com estado interno (mínimo e máximo) em um componente *stateless* com `RangeSlider`. Escreva dois `@Preview`s com valores diferentes.

Exercício 3 — Ciclo de vida

Registre um `Log` em cada callback do ciclo de vida da `MainActivity`. Gire a tela, vá para a tela inicial e volte. Desenhe a sequência observada.

LabMonitor — Etapa 5

Implemente a navegação: **Painel**, **Equipamentos** (lista → detalhe → reserva) e **Ajustes** com `NavigationBar`. O formulário de reserva deve sobreviver à rotação.

Para aprofundar

- Estado e Jetpack Compose: `https://developer.android.com/develop/ui/compose/state`
- Onde elevar o estado: `https://developer.android.com/develop/ui/compose/state-hoisting`
- Efeitos colaterais: `https://developer.android.com/develop/ui/compose/side-effects`
- Ciclo de vida da Activity: `https://developer.android.com/guide/components/activities/activity-lifecycle`
- Navigation 3: `https://developer.android.com/guide/navigation/navigation-3`
- Recipes do Navigation 3: `https://github.com/android/nav3-recipes`

Fim do Módulo 05

Próximo: Módulo 06 — Arquitetura de apps: MVVM, camadas, repositório, casos de uso e injeção de dependência com Hilt.